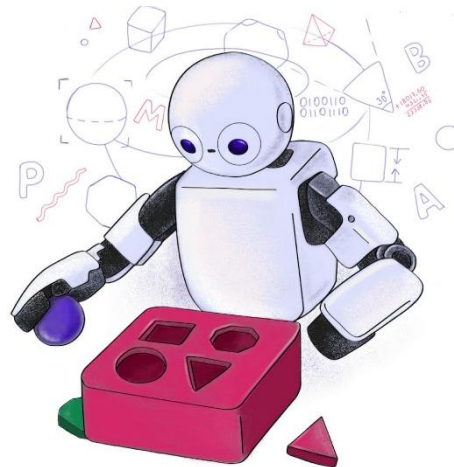


C24 - Inteligência Artificial:

k-Means



Recapitulando

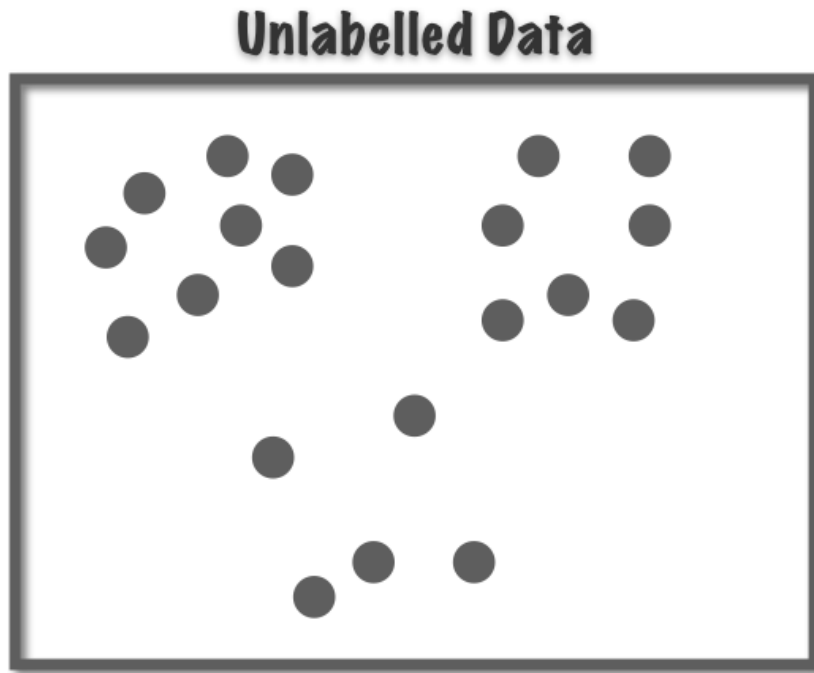
- Até o momento, todos os algoritmos que aprendemos seguiam o paradigma do **aprendizado supervisionado**.
- Hoje, falaremos sobre ***clustering***, que são algoritmos de **aprendizado não-supervisionado**.
 - **Não temos rótulos**, apenas atributos.
- Eles visam criar **agrupamentos** de dados (chamados de ***clusters*** ou **grupos**) segundo seu **grau de semelhança**.
- Nesta aula, aprenderemos sobre o algoritmo chamado de **k-Médias** (ou ***k-Means***, em inglês) que é um dos algoritmos mais simples de ***clustering***.
 - OBS.: Agrupamentos, grupos ou *clusters* são sinônimos neste contexto.

clustering be like:



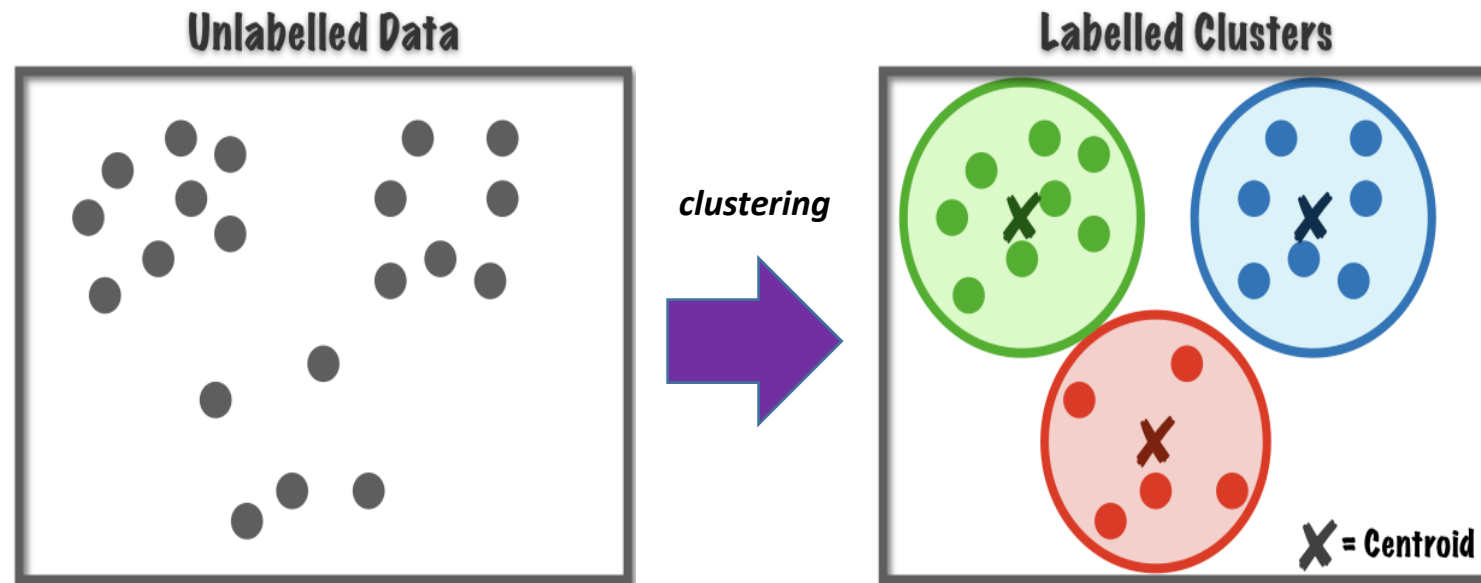
A **clusterização** organiza dados como na figura: cada ponto encontra seu **grupo mais próximo**, formando *clusters*.

Motivação



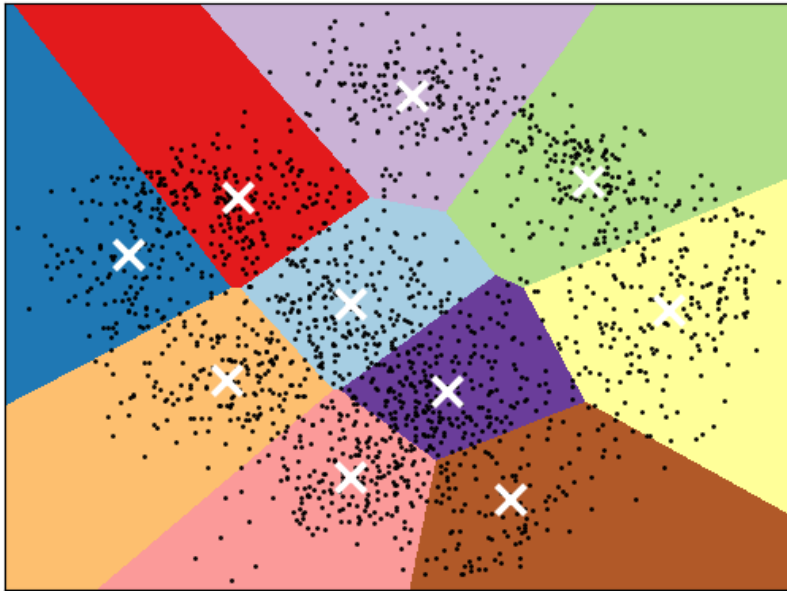
- O que podemos fazer se **não tivermos informações sobre as classes** (i.e., rótulos) a que pertencem os exemplos?
- Conseguimos extrair alguma informação útil?

Motivação



- O aprendizado não-supervisionado **extraí estrutura e informações úteis de dados sem rótulos.**
- O *clustering* tenta **reproduzir como humanos organizam o mundo: agrupamos objetos por similaridade.**

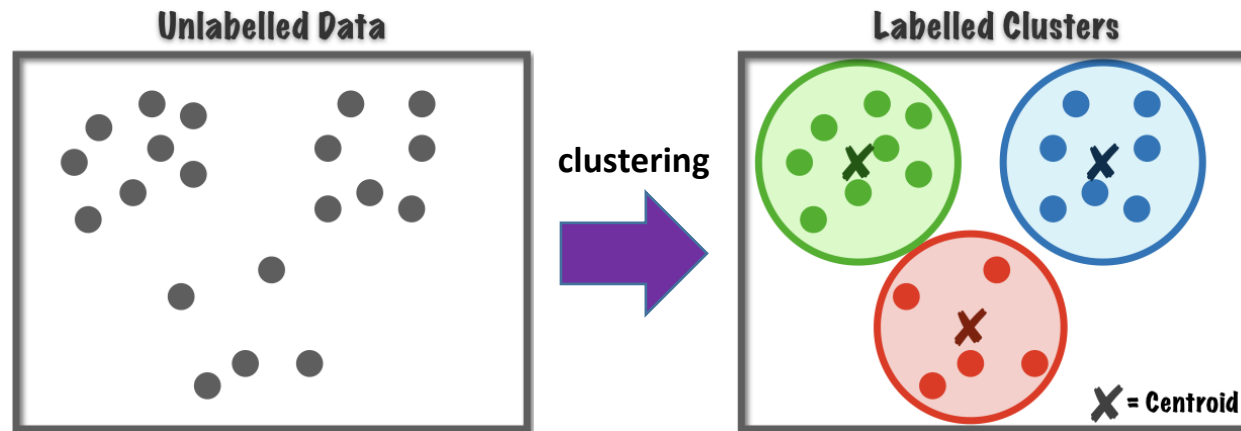
Motivação



- O **clustering** agrupa dados com base em **similaridade** sem precisar de supervisão.
- A ideia mais importante é revelar a **estrutura intrínseca dos dados**:
 - padrões escondidos
 - subgrupos naturais
 - relações não explícitas

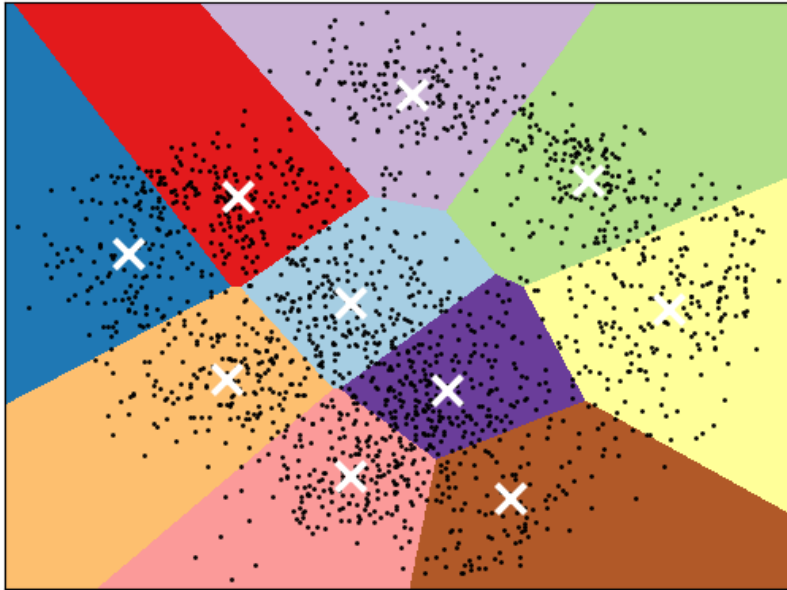
Identificação de *clusters*

- A **tarefa fundamental** do aprendizado não-supervisionado é a **identificação de *clusters***.
- Nessa tarefa, a **entrada** é um conjunto de **vetores de atributo** (i.e., exemplos), **mas sem rótulos**.
- A **saída** são os **centroides** e os **índices dos *clusters*** a que pertencem cada um dos exemplos do conjunto de treinamento.



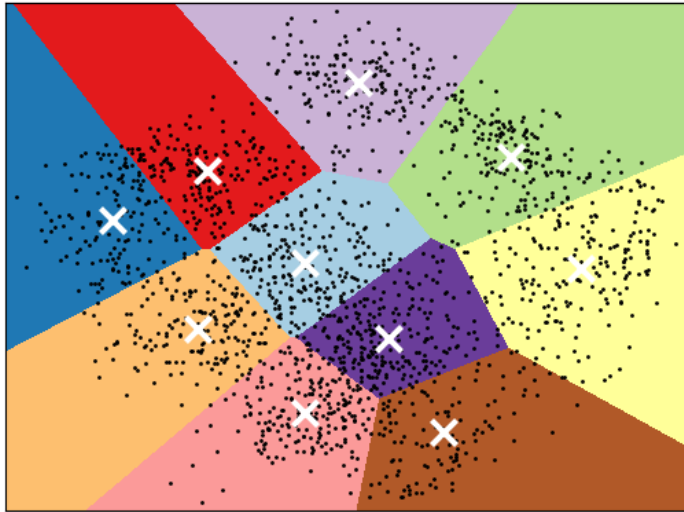
OBS.: A identificação visual de *clusters* em um espaço bi ou tridimensional é fácil, mas em quatro ou mais dimensões isso já não é mais possível. Nesses casos, apenas algoritmos de identificação de *clusters* conseguem agrupar os dados.

Aplicações



- Redução de complexidade e dimensionalidade
 - Em um **conjunto muito grande de dados**, podemos **armazenar os centroides e/ou os índices correspondentes aos vetores de atributo** ao invés de todos os vetores de atributo.
 - Pode ser usado para comprimir imagens.
 - **Vetores de atributo muito longos** podem ser representados por um **vetor com as distâncias a um conjunto de centroides**.

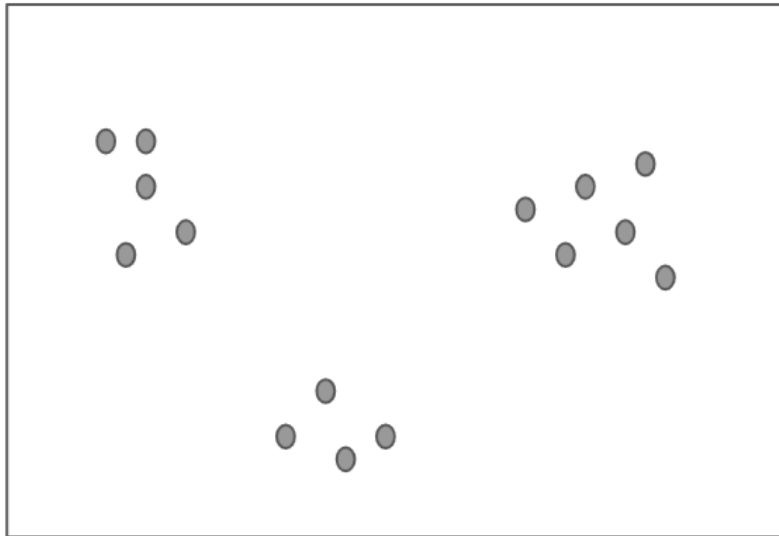
Aplicações



- Detecção de padrões raros
 - detector de comportamento “normal”
 - tudo fora disso é considerado suspeito
- Pré-processamento para outros modelos
 - criar novos atributos
 - e.g., o índice do *cluster*, distâncias até os *clusters*
 - inicializar modelos
 - Muitos algoritmos (e.g., GMM) precisam definir centros iniciais.
 - organizar dados antes do treinamento de modelos de aprendizado supervisionado
 - Identificar subgrupos implícitos (e.g., para treinar um modelo específico para cada subgrupo)
 - Detecção de *outliers*

Como representar os *clusters*?

Unlabelled Data

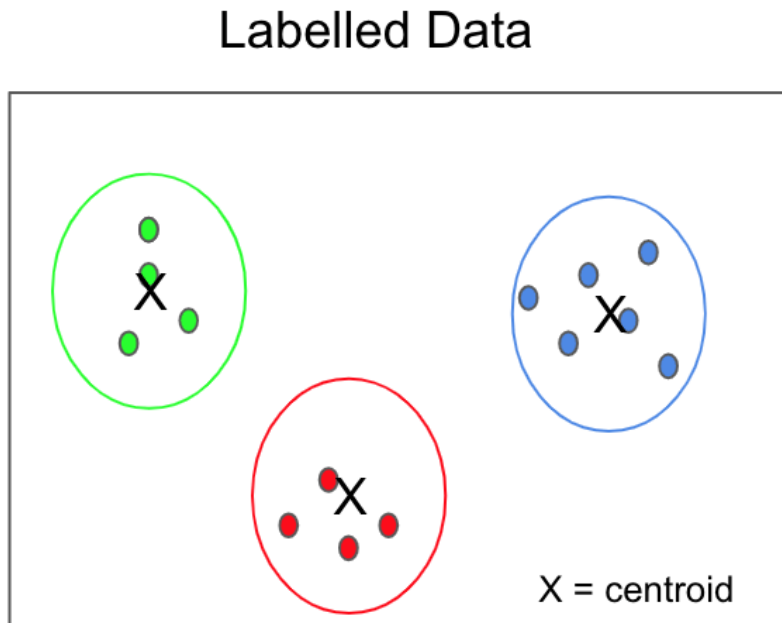


- Para identificar *clusters*, temos que primeiro **decidir como eles serão representados**.
- Existem algumas opções como a **densidade**, **distribuição**, **hierarquia**, etc.
- Porém, a abordagem mais simples usa os **centroides** (i.e., centros) dos *clusters*.
- O algoritmo ***k-Means*** implementa esta abordagem.
- Vejamos então suas características e como funciona.

Centroide

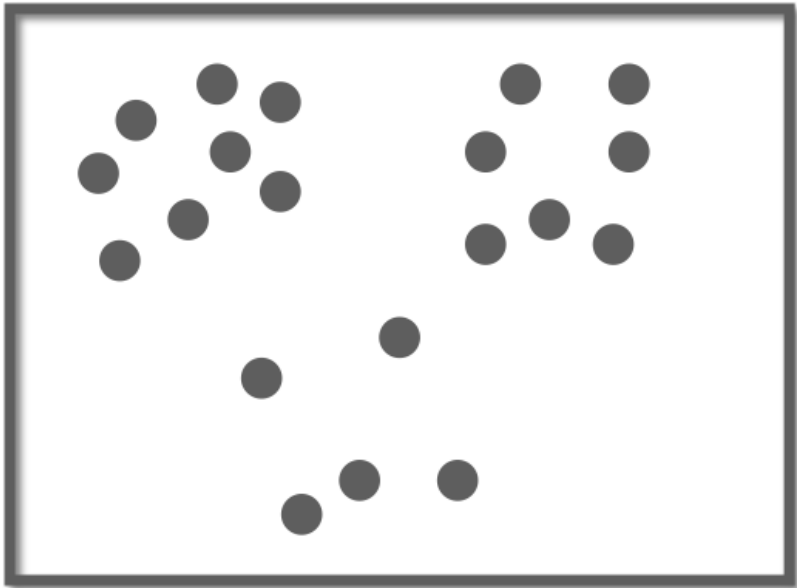
- **Centroide** é o ponto central de um *cluster* de dados.
 - Representa a "**média**" das amostras que pertencem a um *cluster*.
- Os **centroides** podem ser usados como
 - centros para inicializar GMMs, redes neurais Bayesianas ou de funções de base radial (RBF),
 - estimativas de valores de atributos ausentes.
 - forma de comprimir dados usando apenas o índice do centróide mais próximo.
 - *feature engineering*: o índice do centróide (ID) e a distância até os centróides podem ser novos atributos.
 - forma de detectar anomalias: distância alta até o centróide pode indicar *outlier* ou dado que foge do padrão (evento raro).
 - forma de agrupar dados sem classes em problemas semi-supervisionados: os *clusters* são usados como classes e, conseqüentemente, rotular os dados.

Como representar os *clusters*?



- Por exemplo, suponhamos os seguintes **vetores de atributos** em um espaço bidimensional formado por x_1 e x_2 : (2, 5), (1, 4), (3, 6).
- Se eles pertencem a um *cluster*, seu **centroide** é representado pelo vetor (2, 5), pois
 - A média do primeiro atributo é $\frac{2+1+3}{3} = 2$.
 - A média do segundo atributo é $\frac{5+4+6}{3} = 5$.
- **OBS.:** Em geral, **aplicamos clusterização apenas a atributos numéricos**.
 - Existem algoritmos de clusterização específicos para dados categóricos: *k-modes* e *k-prototypes*.

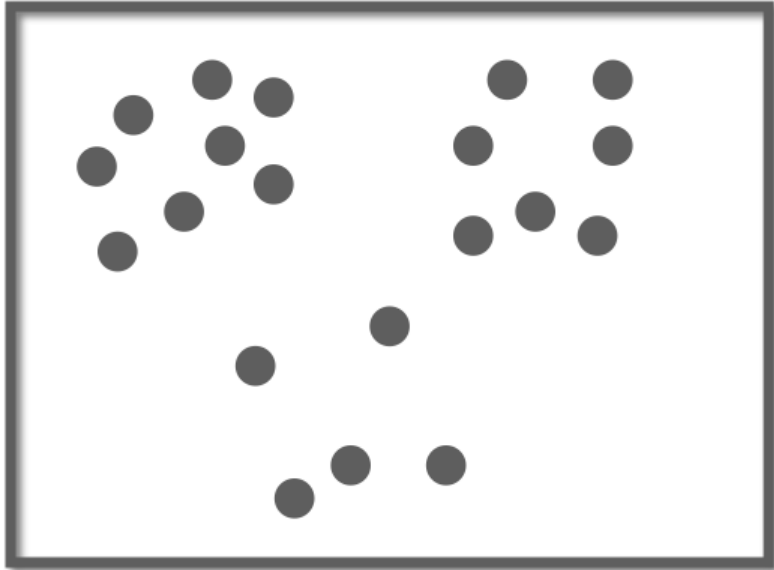
Quantos devem ser os *clusters*?



- Os *clusters* não devem se sobrepor.
- Cada amostra deve pertencer a um e apenas um *cluster*.
- Porém, dentro do mesmo *cluster*, as amostras devem estar relativamente próximas umas das outras e distantes das amostras dos outros *clusters*.
- Aí surge uma dúvida.

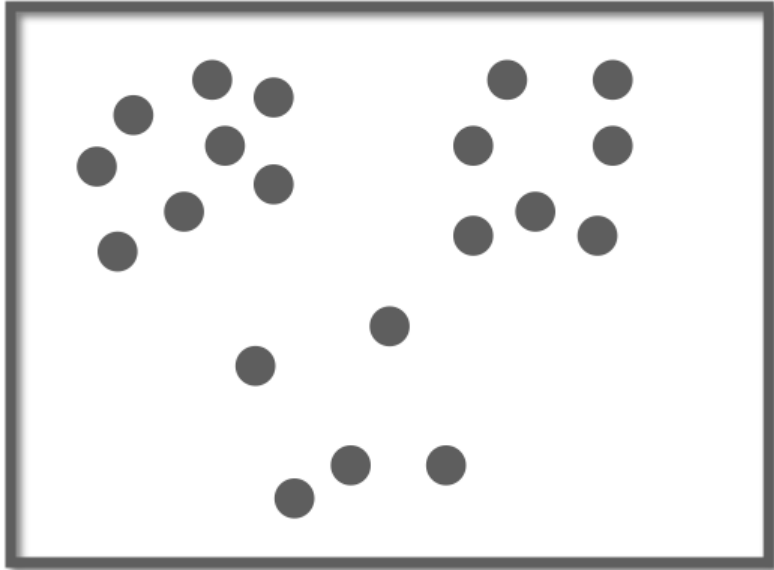
Quantos *clusters* um conjunto de dados contém?

Quantos devem ser os *clusters*?



- Na figura conseguimos identificar visualmente três *clusters*.
- No entanto, o número de opções existentes não se limita a essa única possibilidade:
 - em um extremo, todo o conjunto de dados pode ser pensado como formando um grande *cluster*;
 - no outro extremo, cada exemplo pode ser visto como representando seu próprio *cluster* de um único exemplo.

Quantos devem ser os *clusters*?



- **Implementações** práticas geralmente evitam esse problema **pedindo** ao **usuário** que **forneça o número de *clusters***.
- Existem diversos métodos para **otimizar o hiperparâmetro que define o número de *clusters***, dentre eles, o mais usado é o **método do cotovelo**.
- Veremos como ele funciona em breve.
- Porém, antes, precisamos entender como os *clusters* são criados.

Medindo distâncias

- Algoritmos para identificação de *clusters* precisam de uma **métrica para avaliar a distância entre uma amostra e o centroide** de um *cluster*.
- Uma forma de fazer isso é usar a **distância euclidiana** entre os dois vetores

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^K (x_i - y_i)^2},$$

onde K é o número de atributos e x_i e y_i são os i -ésimos elementos dos vetores tendo sua distância calculada.

- Outras medidas de distância: Minkowski, cosseno, Manhattan e Mahalanobis.

A qual *cluster* um exemplo deve pertencer?

- Vamos supor que existam N *clusters* cujos centroides são denotados pelo vetor, $\mathbf{c}_i, \forall i \in (1, N)$.
- Uma amostra $\mathbf{x}$ tem uma certa distância $d(\mathbf{x}, \mathbf{c}_i)$ para cada centroide.
- Se $d(\mathbf{x}, \mathbf{c}_p)$ é a menor dessas distâncias, então, colocarmos $\mathbf{x}$ como pertencente ao centroide $\mathbf{c}_p$, ou seja, o p -ésimo *cluster*.
- Portanto, escolhemos a **menor distância** para **definir a qual *cluster* um exemplo pertence**.
- Agora, veremos como funciona o algoritmo *k-Means*.

k-Means

- O "*k*" é um hiperparâmetro que define o número de *clusters*.
- O pseudocódigo do algoritmo é mostrado abaixo.

Entradas: conjunto de exemplos e número de *clusters*, *k*.

1. Defina *k* centroides iniciais (geralmente, escolhidos do *dataset* de forma aleatória).

2. Repita

- a) Calcule a distância de cada exemplo, *x*, para cada um dos *k* centroides.
- b) Atribua cada exemplo, *x*, ao *cluster* mais próximo (menor distância).
- c) Calcule o novo centroide de cada *cluster**

3. Enquanto as posições dos centroides continuarem mudando.

*os novos centroides são calculados como a média aritmética (centro geométrico) das amostras pertencentes a cada *cluster*.

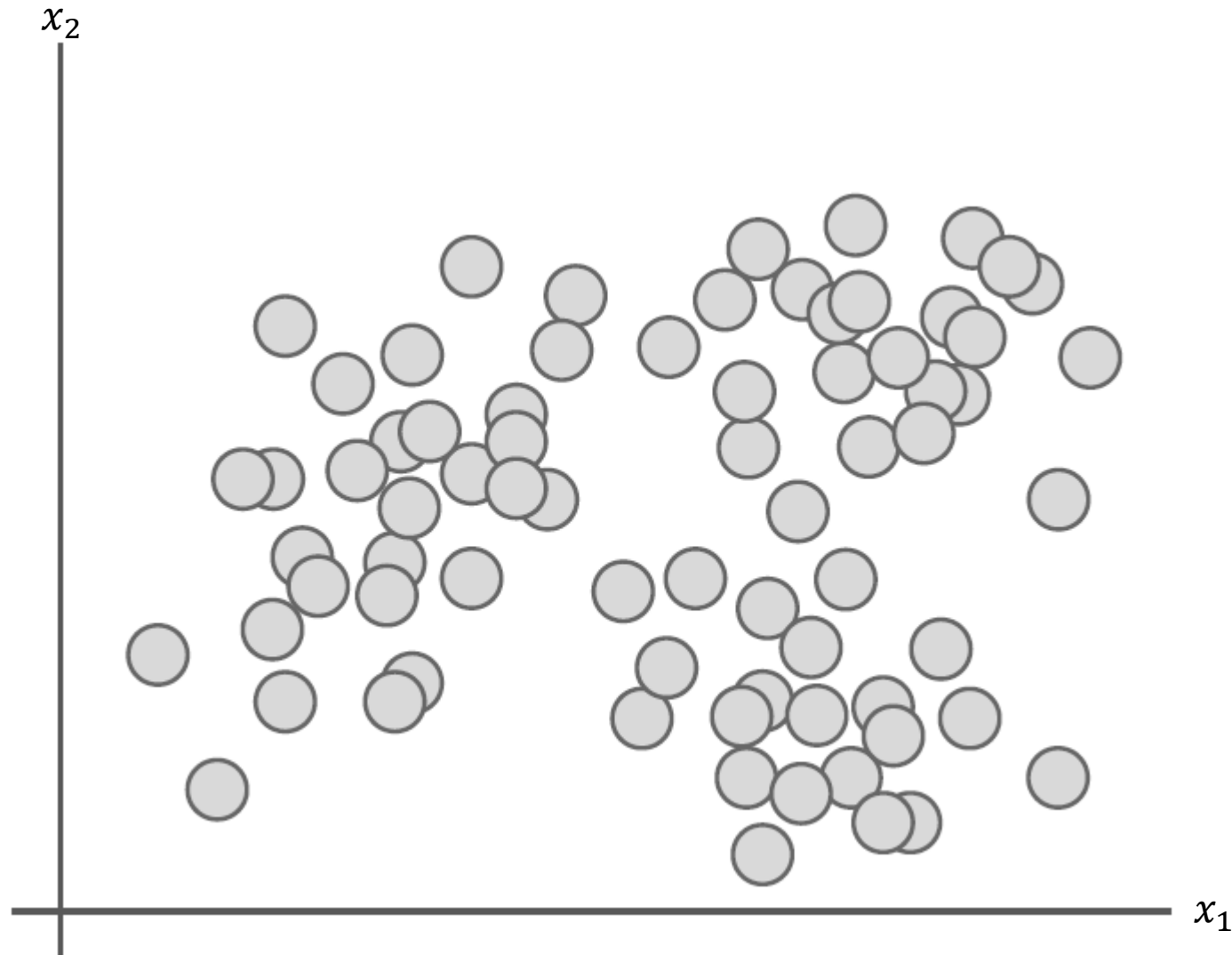
- O algoritmo garantidamente chega a uma situação em que cada exemplo se encontra no *cluster* mais próximo, de modo que, a partir deste momento, os centroides não mudem mais.

Vejam os um exemplo

Funcionamento do *k-Means*

1. Define-se o número de clusters, $k = 3$

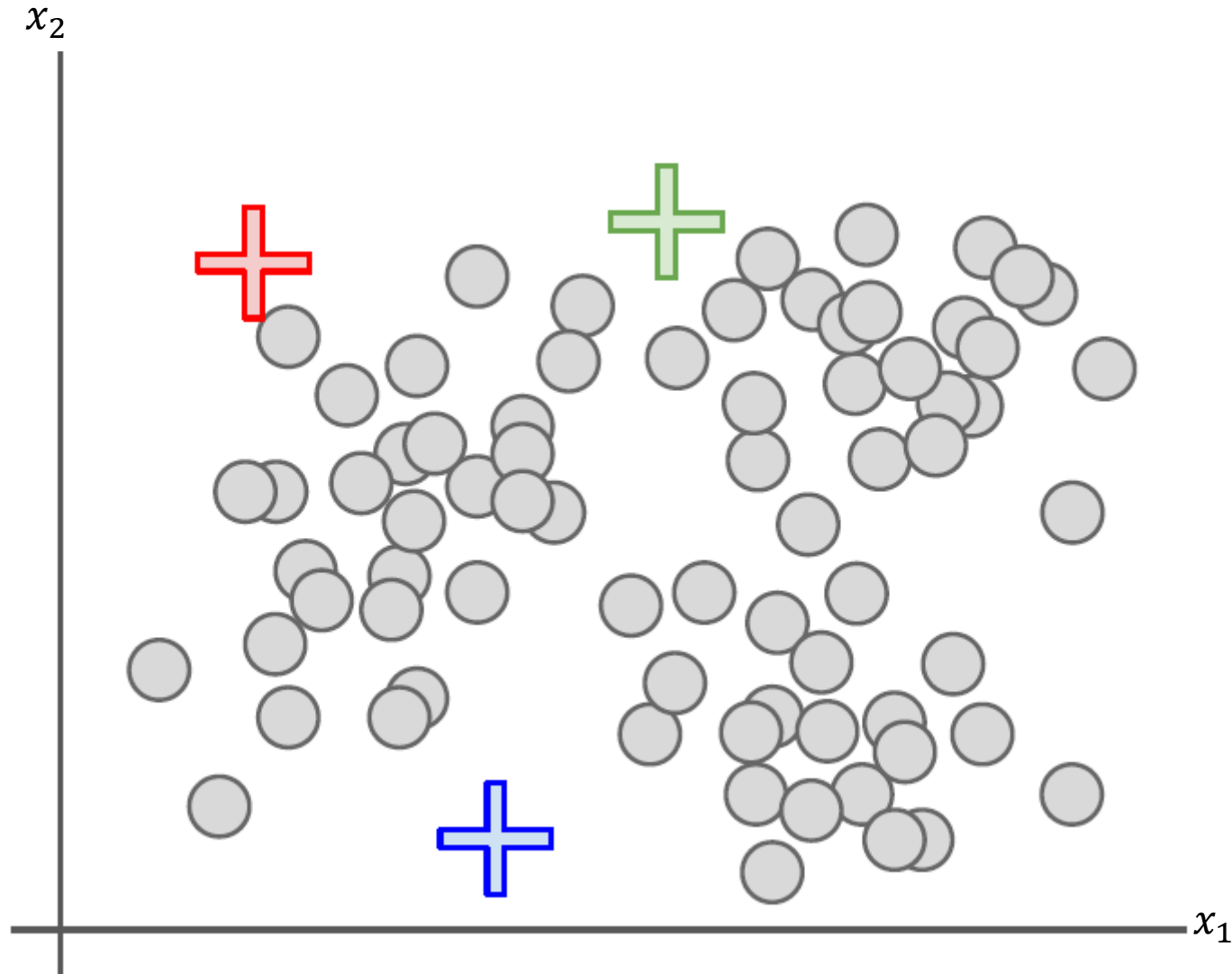
- $k = 3$ pode não ser o valor ideal.
- Para encontrar o valor ideal, pode-se usar o método do cotovelo.



Funcionamento do *k-Means*

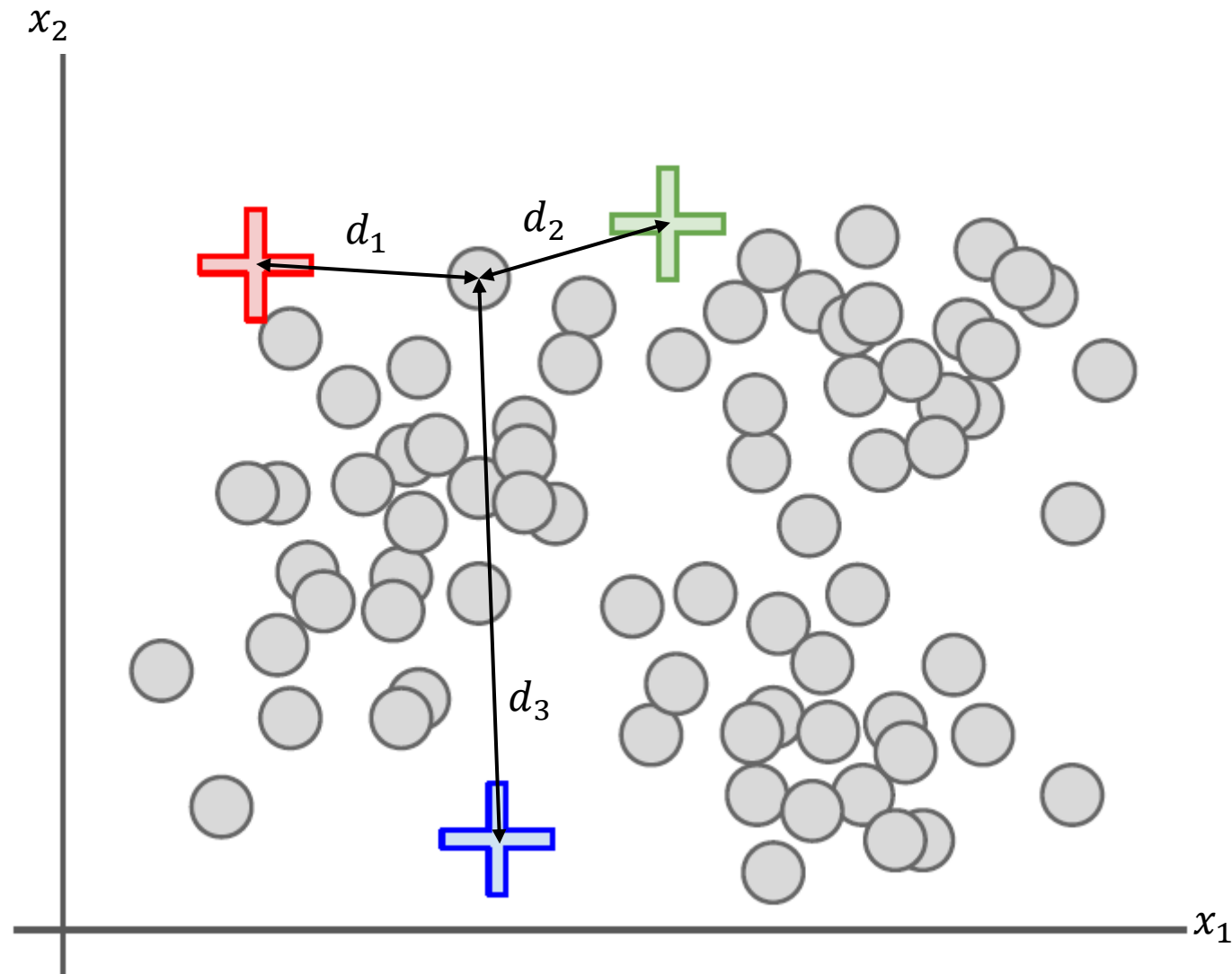
2. Escolhe-se os $k = 3$ centroides (cruzes coloridas) para cada cluster de forma **aleatória**.

- No caso desse exemplo, os centroides são vetores bidimensionais.



Funcionamento do *k-Means*

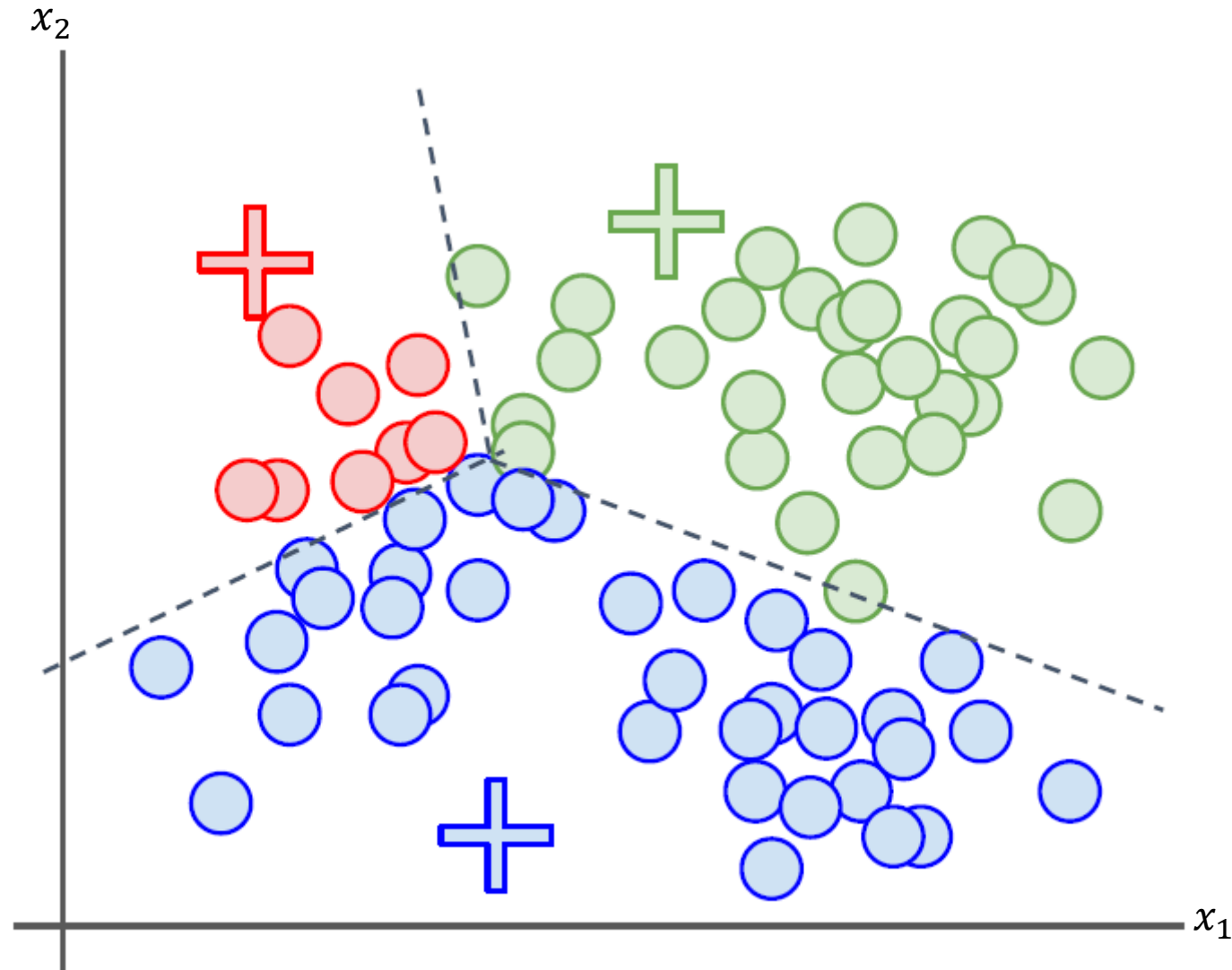
3. Calcula-se a **distância** de cada amostra (círculos cinzas) para cada um dos 3 centroides.



Funcionamento do *k-Means*

4. Atribui-se cada amostra ao centroide mais próximo com base na distância euclidiana.

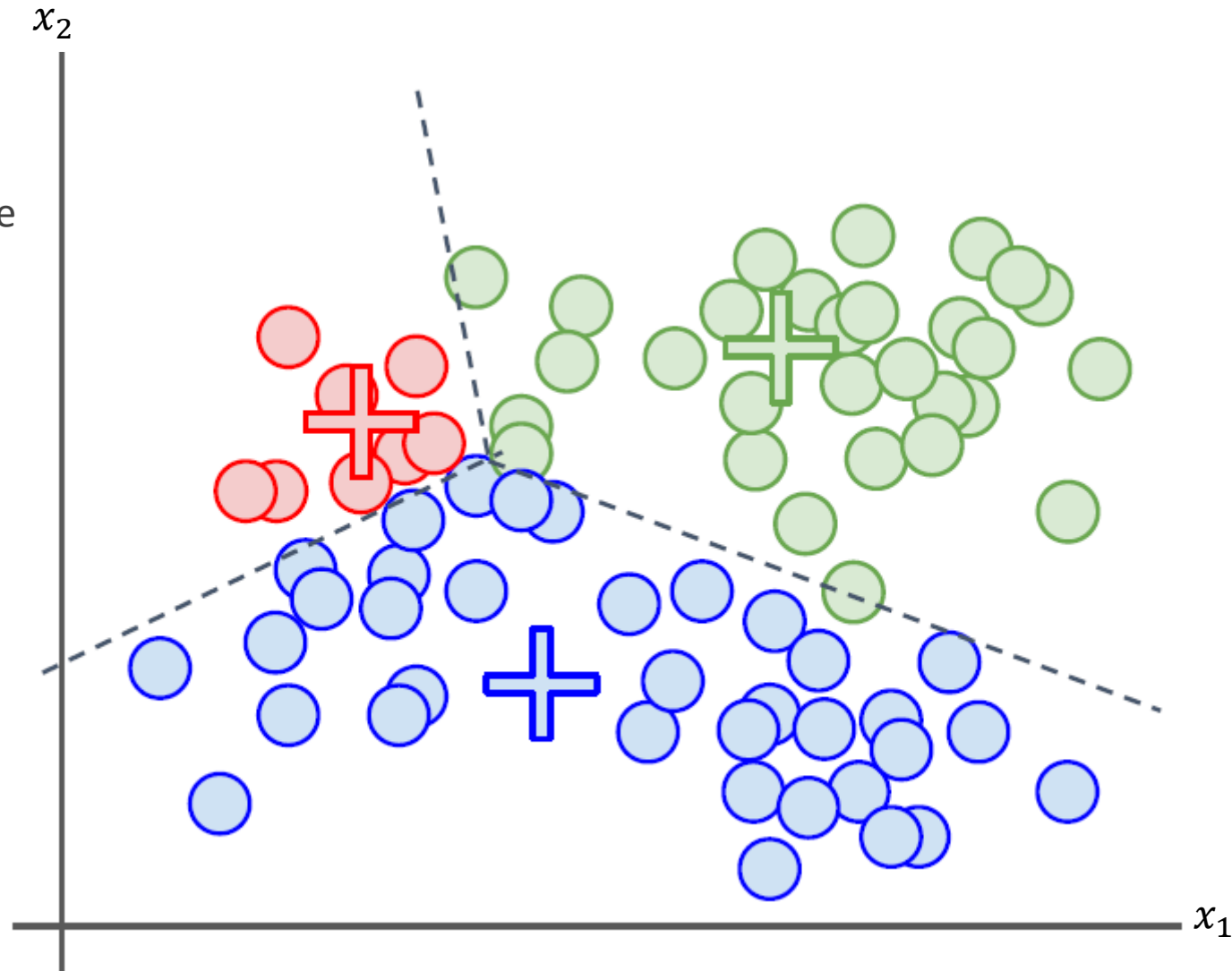
Cada centroide define uma região.



Funcionamento do *k-Means*

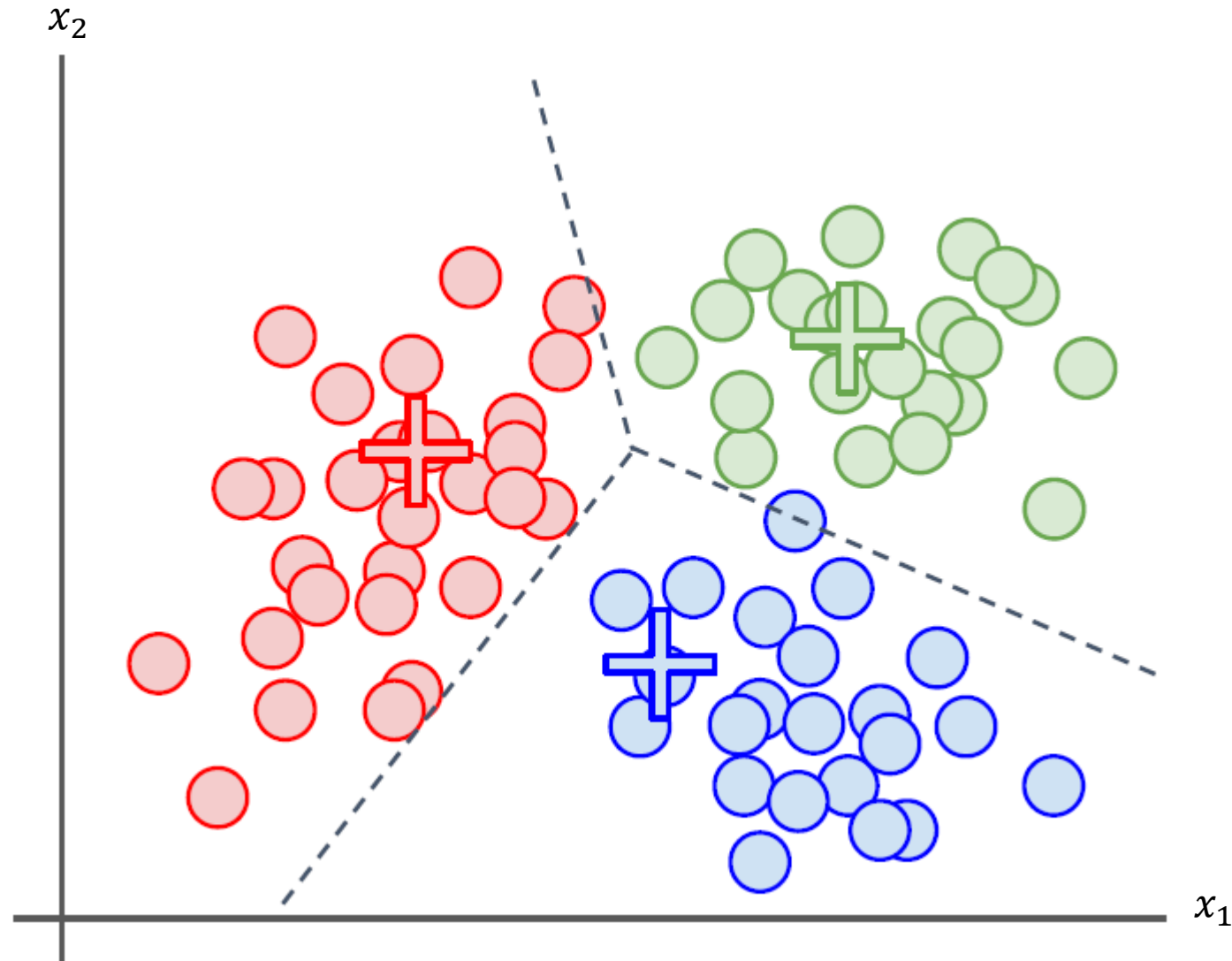
5. Calcula-se o novo centróide de cada cluster usando-se a média das amostras.

Os centróides são os **centros geométricos** de cada *cluster*.



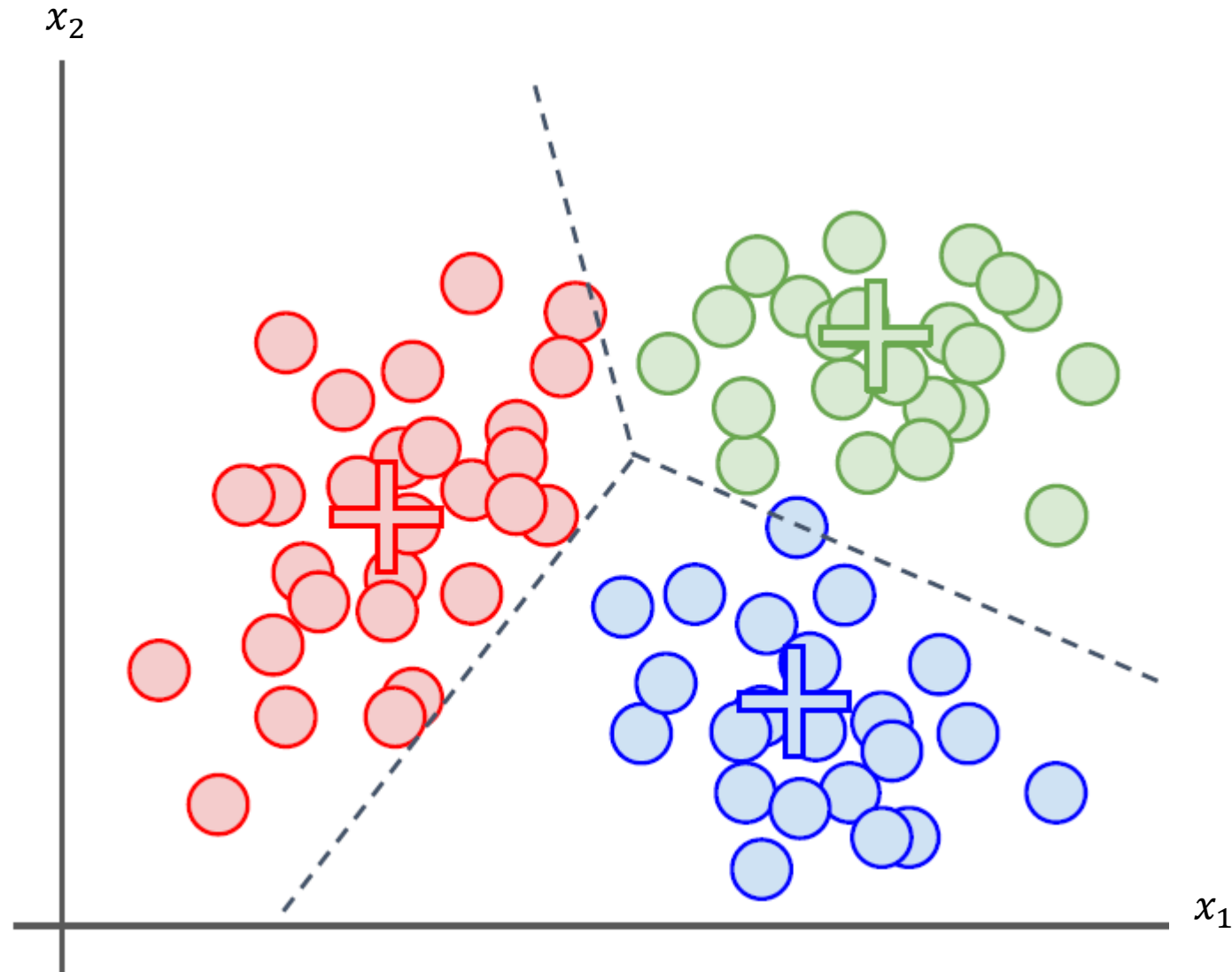
Funcionamento do *k-Means*

6. Repete-se os passos 3, 4 e 5.



Funcionamento do *k-Means*

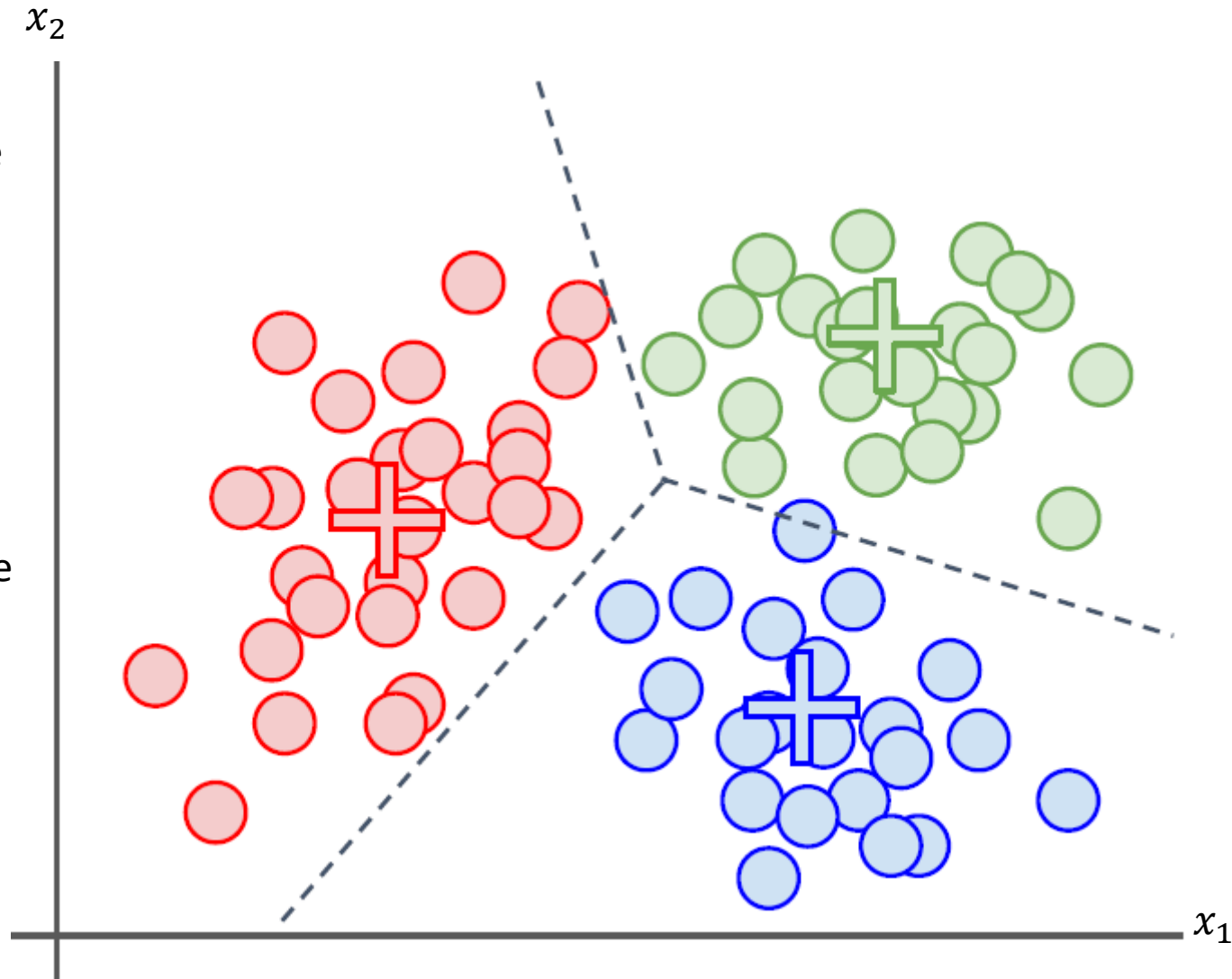
7. Repete-se os passos 3 e 4.



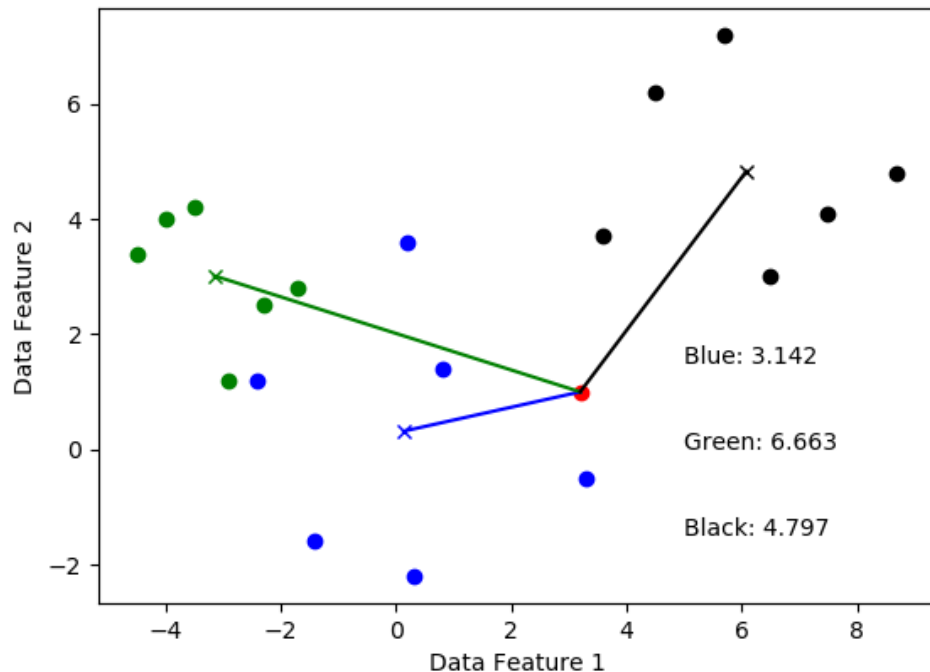
Funcionamento do *k-Means*

Repete-se os passos 3 à 5 até que um dos casos abaixo ocorra:

1. Os centroides não mudem mais (ou mudem muito pouco),
2. A soma das distâncias entre as amostras e o centroide correspondente seja minimizada (e permaneça praticamente constante),
3. O número máximo de iterações seja atingido.

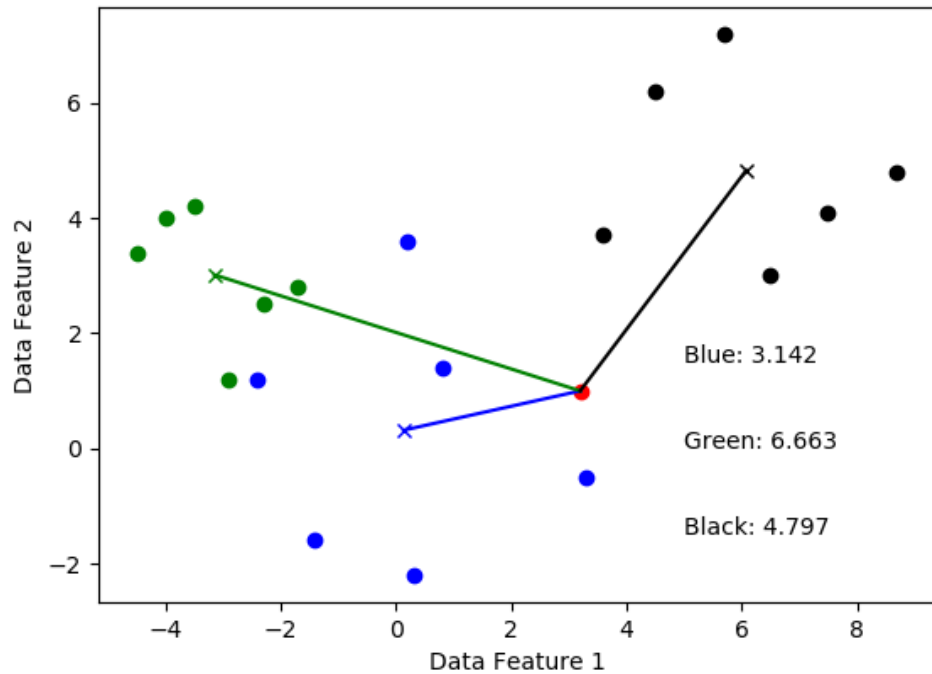


Como inicializar os centroides?



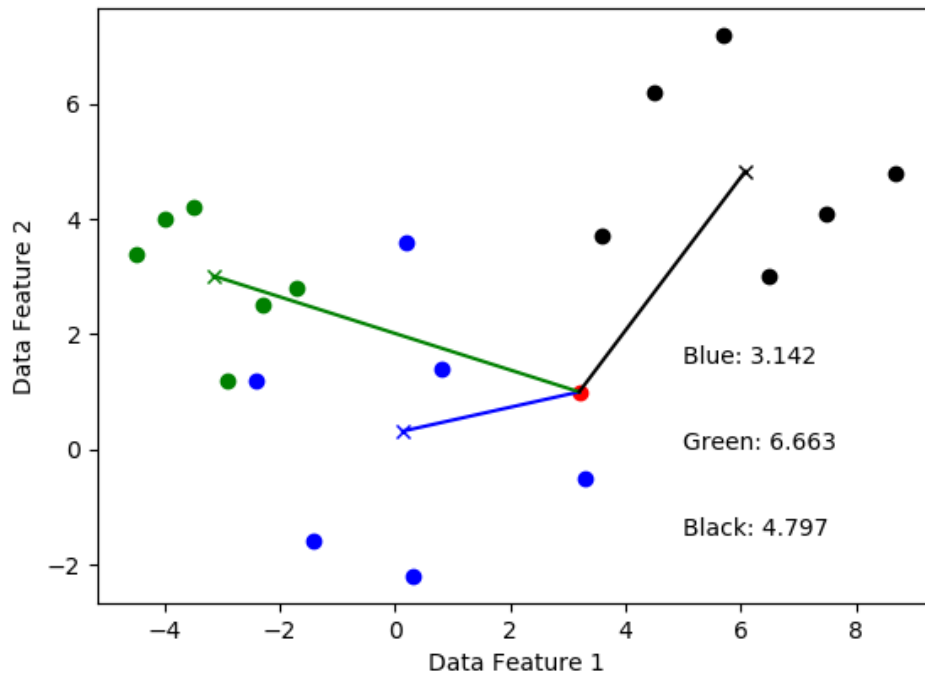
- A inicialização mais simples é **escolher k exemplos de treinamento aleatórios** como os centroides iniciais.
- Porém, existem algumas desvantagens.
- Se os centroides iniciais mudam:
 - os *clusters* iniciais podem mudar completamente
 - fazendo com que o algoritmo convirja para *clusters* diferentes a cada execução.
- Ou seja, **cada inicialização aleatória** pode resultar em **centroides finais diferentes**.
 - O desempenho pode variar com a inicialização.

Como inicializar os centroides?



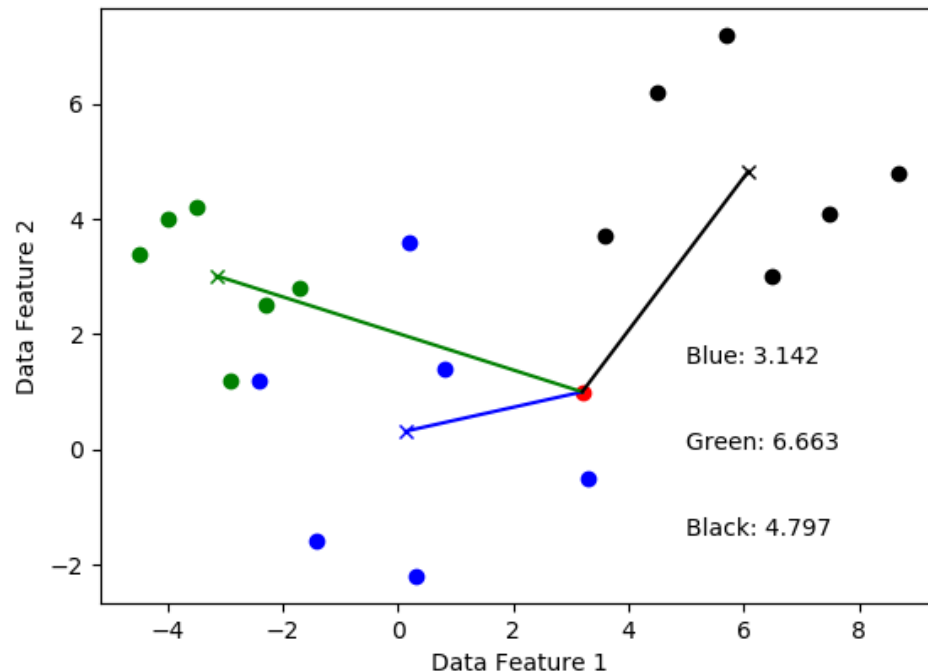
- O número de **transferências** de um exemplo **de um *cluster* para outro** **depende dos centroides iniciais**.
- Se os **centroides iniciais** forem **muito próximos**, o tempo (de treinamento) para encontrar os melhores *clusters* pode ser alto (i.e., muitas transferências).
- Por outro lado, se os **centroides iniciais** já forem **perfeitos**, **nenhum exemplo é transferido para outro *cluster*** e o algoritmo é encerrado.

Como inicializar os centroides?



- Portanto, a **inicialização é importante** no sentido de que **um ponto de partida melhor** garante que **solução seja encontrada mais rápido**.
- Ou seja, o **tempo de convergência (e o desempenho)** do algoritmo **depende da inicialização**.
- O algoritmo é **altamente sensível à escolha inicial dos centroides**.

Como inicializar os centroides?



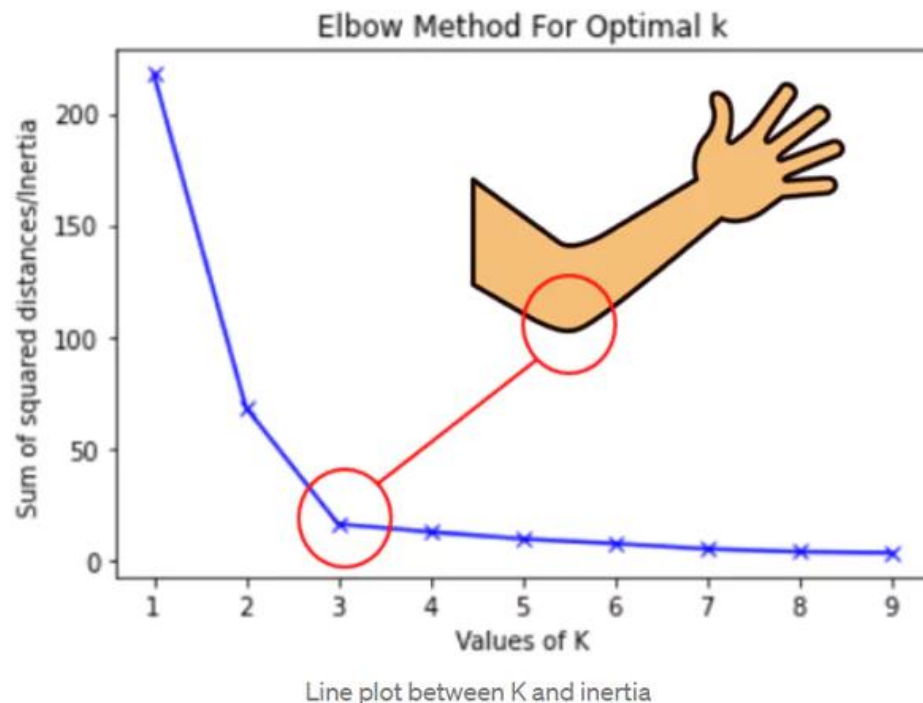
- Em resumo, todo o processo depende do ponto de partida.
- Portanto, como encontramos os melhores *clusters*?

Inicializamos o algoritmo aleatoriamente diversas vezes e escolhemos o **melhor resultado***.

*O resultado que minimiza a distância *intraclusters*.

Como definimos o número de
clusters?

Método do cotovelo



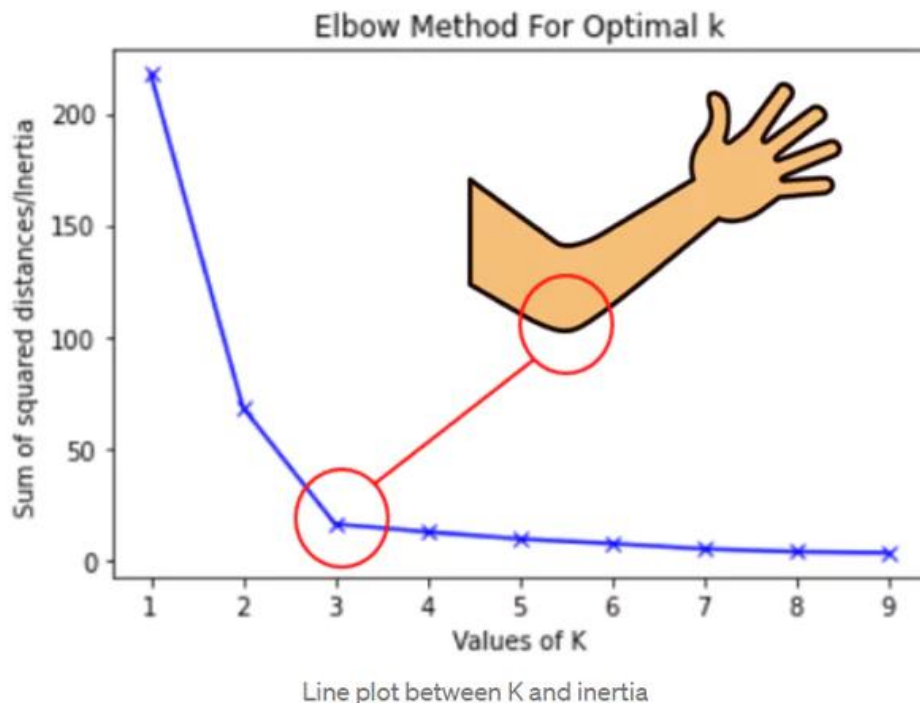
- O método busca o ponto (i.e., k) onde a **melhoria trazida pelo aumento de *clusters* começa a diminuir significativamente**.
- Ou seja, aumentar k sempre melhora o desempenho, mas chega um ponto em que o ganho é **mínimo**.
- Esse ponto é chamado de “**cotovelo**”.
- Que métrica usamos para plotar a curva e encontrar o cotovelo?

Soma dos quadrados intra-*clusters*

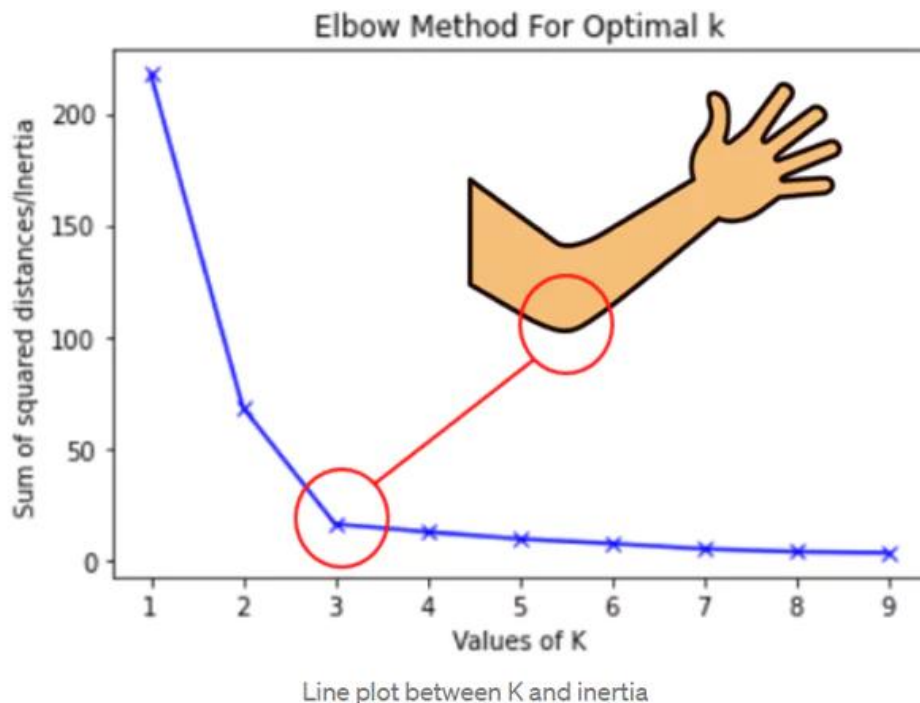
- Usa-se a soma dos quadrados intra-*clusters*, também conhecida como **inércia**

$$I(k) = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

onde μ_i é o centroide do i -ésimo *cluster*, C_i é conjunto de amostras no i -ésimo *cluster* e $\|x - \mu_i\|^2$ é a distância até μ_i .

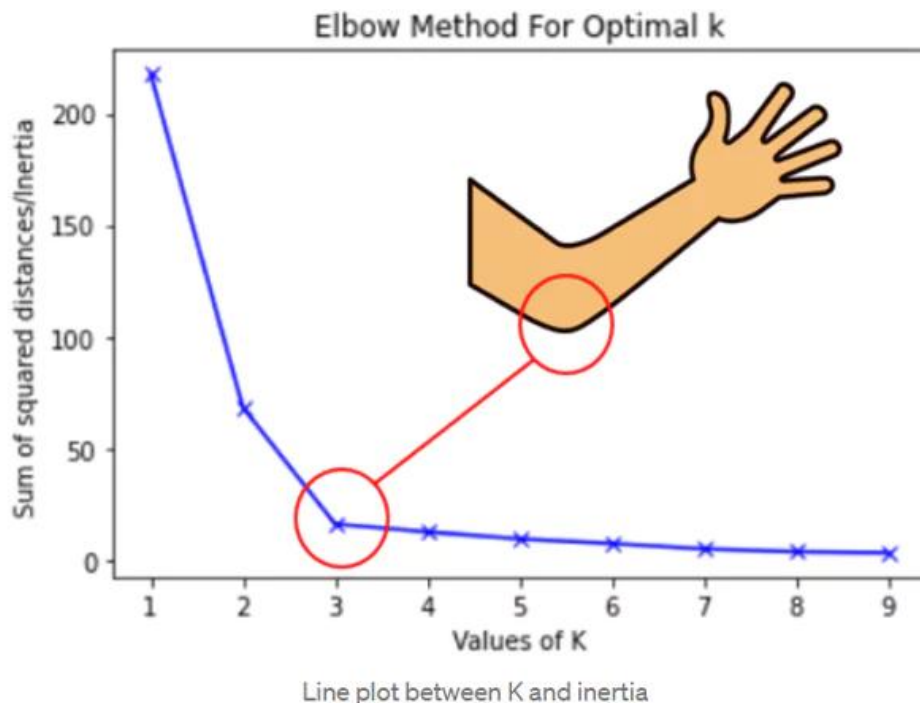


Soma dos quadrados intra-*clusters*



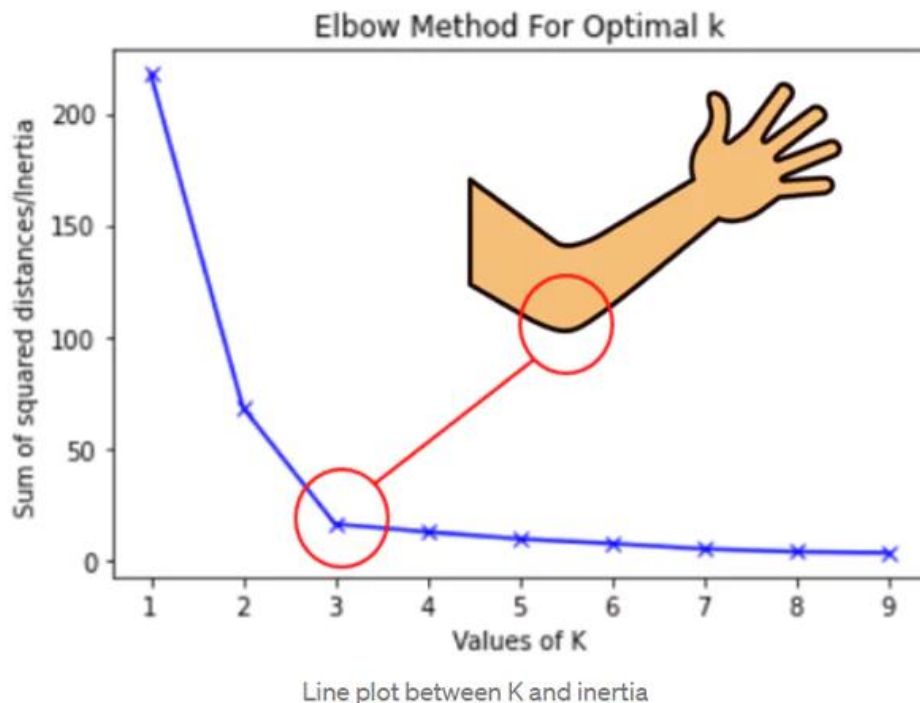
- Ela mede o quão “**compactos**” são os *clusters*.
 - quanto menor → mais próximas as amostras estão de seus centroides.
- No limite, quando k igual a N , a inércia é 0 (mínima possível), mas o **modelo será extremamente complexo** e poderá se **sobreajustar** aos dados de treinamento.
- Então como encontramos o cotovelo?

Soma dos quadrados intra-*clusters*



- Calcula-se e plota-se o valor da inércia para um intervalo de valores de k .
- A curva resultante quantifica **o quanto próximos os pontos de um *cluster* estão do seu centroide.**
- Quanto **menor a inércia, mais coesos são os *clusters*:**
 - os pontos dentro de um *cluster* estão fortemente agrupados, **próximos entre si e do centroide.**

Soma dos quadrados intra-*clusters*

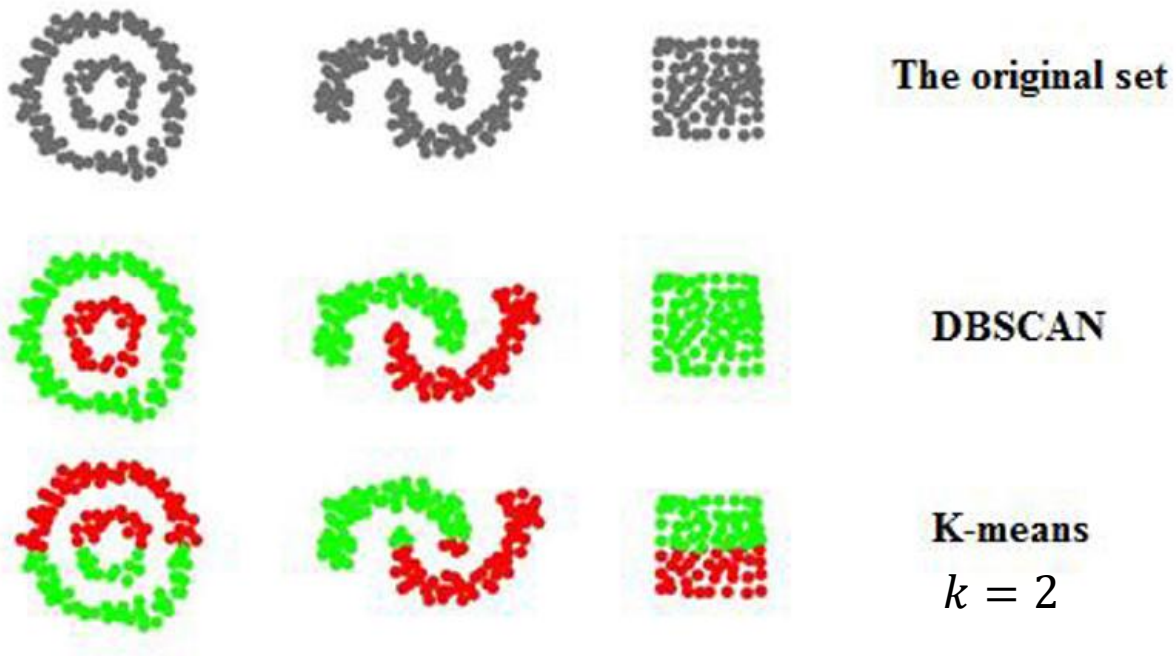


- Assim, o valor ideal de k é encontrando localizando-se o **cotovelo na curva**.
- Quando o cotovelo não é visível (transição suave), uma técnica comum é traçar uma **reta conectando os pontos extremos da curva** e identificar o **ponto mais distante** dessa reta como o "**cotovelo**" ideal.

Outros algoritmos de *clustering*

- *Density-based spatial clustering of applications with noise* (DBSCAN): o agrupamento é baseado em **densidade**, enquanto o *k-means* é baseado em **centróides**.
- **Não** é necessário definir o número de *clusters* previamente.
- Ele busca **regiões de alta densidade** para criar *clusters*.
- Possui dois parâmetros:
 - ε (epsilon): raio da vizinhança;
 - MinPts: número mínimo de pontos dentro desse raio para caracterizar uma região densa.
- **Problema:** um único valor de ε raramente funciona bem quando diferentes *clusters* possuem densidades diferentes.

Outros algoritmos de *clustering*

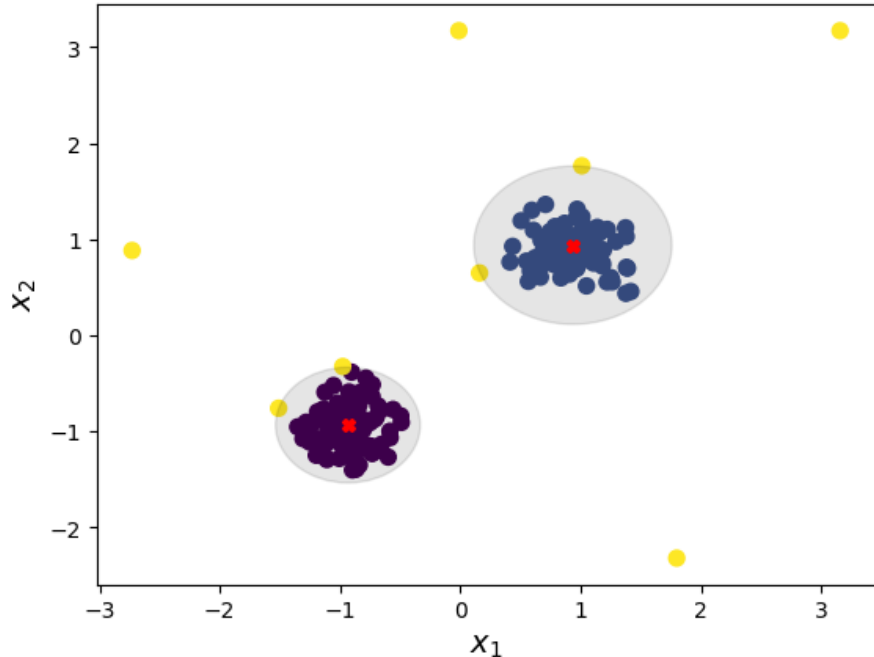


- Enquanto o *k-means* responde a pergunta
*Onde devo colocar k centróides para **minimizar a distância média** dos pontos?*
- O DBSCAN responde a pergunta:
*Quais regiões do espaço possuem **densidade suficiente** para serem **consideradas clusters**?*
- ***k-means*** funciona bem com **clusters** **aproximadamente esféricos**, já o **DBSCAN detecta formas arbitrárias**.

Outros algoritmos de *clustering*

- Hierarchical DBSCAN: É uma extensão do DBSCAN que constrói uma **hierarquia de densidades**.
 - Minimiza o problema do DBSCAN usando uma grande faixa de valores de densidade.
 - Procura regiões densas em múltiplos níveis de densidade e seleciona os agrupamentos mais estáveis.
- Misturas de Gaussianas: Assume que os dados foram gerados por uma mistura de várias **distribuições gaussianas**, cada uma representando um *cluster*.
 - Método probabilístico que fornece probabilidades de pertencimento aos *clusters*.
- Documentação da biblioteca *scikit-learn* sobre clusterização:
 - <https://scikit-learn.org/stable/modules/clustering.html>

Aplicação: detecção de anomalias com *k-means*



- Nessa aplicação, encontramos os **centroides e a área dos *clusters***.
- A **área** é definida por um valor de **raio** que faça com que **uma dada porcentagem* das amostras** tenham **distância** ao centroide mais próximo **menor ou igual a esse valor**.
- Assim, uma amostra é classificada como **anomalia** se a **distância entre ela e o centroide** mais próximo **for maior do que o raio**.
 - Ou seja, se ela estiver fora da área do *cluster*.

* Em geral, usa-se um valor acima de 95%.

Exemplo

- [Exemplo: kmeans.ipynb](#)

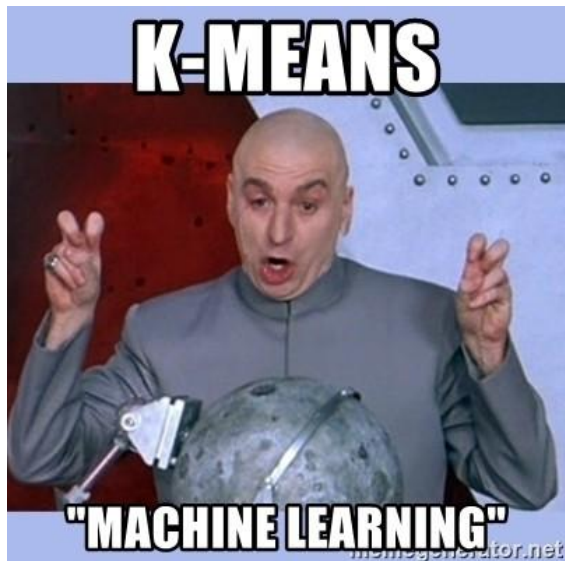


Lista de exercícios

[Lista #13 – kmeans](#)

Perguntas?

Obrigado!



k-means be like:



K-MEANS CLUSTER

